TD2 — Premiers scripts Shell

Systèmes d'exploitation

Axel Guérin

Objectifs du TD

À la fin de cette séance, vous devrez être capables de :

- écrire un premier script Shell ;
- exécuter un script avec Bash ;
- rendre un fichier exécutable ;
- comprendre le rôle du *shebang* ;
- utiliser des variables ;
- transmettre des arguments à un script ;
- exécuter un programme Python depuis le terminal ;
- utiliser un script Shell pour lancer un programme Python.

Objectif principal

Utiliser le Shell pour **automatiser l'exécution de plusieurs commandes et programmes**.

1 — Quelques rappels

Lors du TD précédent, nous avons utilisé des commandes comme :

```
pwd
ls
cd
mkdir
touch
cp
mv
rm
```

Jusqu'à présent, nous avons exécuté les commandes **une par une**.

Question

Comment demander au système d'exécuter **automatiquement une suite de commandes** ?

Pourquoi écrire un script ?

Supposons que nous devions régulièrement :

1. afficher la date ;
2. afficher notre position ;
3. créer un répertoire ;
4. lancer un programme Python ;
5. afficher un message lorsque le traitement est terminé.

Nous pourrions saisir toutes les commandes à chaque fois.

Ou...

écrire une seule fois ces commandes dans un fichier.

2 — Qu'est-ce qu'un script Shell ?

Un **script Shell** est un fichier texte contenant une suite de commandes.

Par exemple :

`premier_script.sh`

L'extension :

`.sh`

est couramment utilisée pour identifier les scripts Shell.

Le script permet d'exécuter automatiquement plusieurs commandes dans un ordre déterminé.

Créer notre premier script

Créez un fichier :

```
touch premier_script.sh
```

Ouvrez-le avec l'éditeur de votre choix et écrivez :

```
#!/bin/bash

echo "Bonjour !"
echo "Nous executons notre premier script."

pwd
date
```

Enregistrez ensuite le fichier.

La commande echo

Nous venons d'utiliser :

```
echo "Bonjour !"
```

La commande `echo` affiche du texte dans le terminal.

Essayez directement :

```
echo "Bonjour Linux"
```

Puis :

```
echo "Je suis $USER"
```

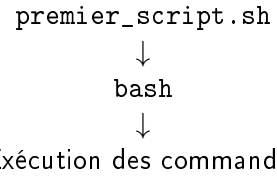
Nous reviendrons bientôt sur la notation `$USER`.

3 — Exécuter notre script

Notre fichier contient des commandes Bash.
Nous pouvons demander à Bash de les interpréter :

```
bash premier_script.sh
```

Bash lit le fichier puis exécute les commandes les unes après les autres.



Shell, Bash et script

Attention à ne pas confondre :

- **Shell** programme permettant notamment d'interpréter des commandes ;
- **Bash** un Shell très utilisé sous Linux ;
- **Script** fichier contenant les commandes que le Shell doit interpréter.

$$\text{Script} \longrightarrow \text{Bash} \longrightarrow \text{Commandes}$$

Un parallèle avec Python

Vous connaissez déjà un fonctionnement similaire.
Pour exécuter :

```
programme.py
```

nous pouvons écrire :

```
python3 programme.py
```

Pour notre script Shell :

```
bash programme.sh
```

Fichier	Interpréteur
programme.py	python3
programme.sh	bash

4 — Exécuter directement le script

Essayons maintenant :

```
./premier_script.sh
```

Selon les permissions du fichier, vous pouvez obtenir :

```
Permission denied
```

Pourquoi ?

Un fichier n'est pas nécessairement **autorisé à être exécuté**.

Ajouter le droit d'exécution

Nous pouvons ajouter le droit d'exécution avec :

```
chmod +x premier_script.sh
```

Puis :

```
./premier_script.sh
```

Le système peut maintenant exécuter directement notre fichier.

À retenir

`chmod +x` permet ici d'ajouter le droit d'exécution au fichier.

Pourquoi ./premier_script.sh ?

Nous écrivons :

```
./premier_script.sh
```

et non simplement :

```
premier_script.sh
```

Rappel :

. = répertoire courant

Donc :

./premier_script.sh

signifie :

« exécuter premier_script.sh situé ici »

5 — À quoi sert la première ligne ?

Notre script commence par :

```
#!/bin/bash
```

Cette ligne est appelée le :

shebang

Elle indique quel programme doit être utilisé pour interpréter le script.

Le rôle du shebang

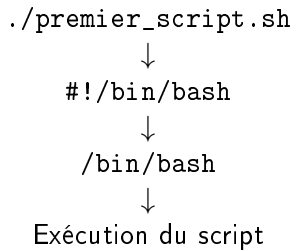
Lorsque nous exécutons :

```
./premier_script.sh
```

le système examine :

```
#!/bin/bash
```

Puis utilise l'interpréteur indiqué.



6 — À vous !

Créez un script nommé :

`informations.sh`

Il doit afficher successivement :

1. le texte `Informations systeme` ;
2. votre nom d'utilisateur ;
3. la date ;
4. votre répertoire courant ;
5. le contenu du répertoire courant ;
6. le texte `Fin du script`.

Rendez ensuite le script exécutable et lancez-le avec :

```
./informations.sh
```

7 — Les variables

Un script peut mémoriser des valeurs dans des **variables**.

Exemple :

```
nom="Alice"
age=18

echo "Bonjour $nom"
echo "Tu as $age ans"
```

Pour récupérer la valeur d'une variable :

$nom

Attention à la syntaxe !

En Bash :

```
nom="Alice"
```

est correct.

Mais :

```
nom = "Alice"
```

est incorrect.

Attention

Il ne faut pas mettre d'espaces autour du symbole = lors de l'affectation d'une variable.

Utiliser une variable

Essayez :

```
#!/bin/bash

prenom="Alice"
matiere="Systemes d'exploitation"

echo "Bonjour $prenom"
echo "Bienvenue en $matiere"
```

Modifiez ensuite les valeurs des variables et exécutez de nouveau le script.

Une variable permet de modifier une information à un seul endroit puis de la réutiliser plusieurs fois.

8 — Donner des informations au script

Jusqu'à présent, les valeurs étaient écrites directement dans le script.

Nous aimerions pouvoir écrire :

```
./bonjour.sh Alice
```

et obtenir :

```
Bonjour Alice !
```

`Alice` est ici un **argument** transmis au script.

Récupérer un argument

Créons :

```
#!/bin/bash

echo "Bonjour $1 !"
```

Puis exécutons :

```
./bonjour.sh Alice
```

Dans le script :

$1

représente le **premier argument**.

Plusieurs arguments

Considérons :

```
#!/bin/bash

echo "Prenom : $1"
echo "Age : $2"
```

Exécution :

```
./informations.sh Alice 18
```

Nous avons :

Variable	Valeur
$1	Alice
$2	18

Combien d'arguments ?

Une autre variable particulière est :

$#

Elle contient le nombre d'arguments transmis.

Exemple :

```
#!/bin/bash

echo "Premier argument : $1"
echo "Deuxieme argument : $2"
echo "Nombre d'arguments : $#"
```

Avec :

```
./script.sh Alice 18
```

$# vaut ici 2.

À vous — Arguments

Créez un script :

`etudiant.sh`

qui s'utilise ainsi :

```
./etudiant.sh Alice Informatique
```

et affiche :

```
Bonjour Alice
Tu etudies Informatique
2 arguments ont ete recus
```

Testez ensuite votre script avec d'autres valeurs.

9 — Et Python ?

Un programme Python est lui aussi un fichier contenant des instructions.

Par exemple :

`programme.py`

Pour l'exécuter sous Ubuntu, nous utiliserons :

```
python3 programme.py
```

Le terminal permet donc de lancer **d'autres programmes**.

Notre premier lancement Python

Créez le fichier :

programme.py

contenant :

```
print("Bonjour depuis Python !")
```

Puis, depuis le terminal :

```
python3 programme.py
```

Vous devriez obtenir :

```
Bonjour depuis Python !
```

10 — Lancer Python depuis un script Shell

Créons maintenant :

lancer.sh

contenant :

```
#!/bin/bash

echo "Debut du traitement"

python3 programme.py

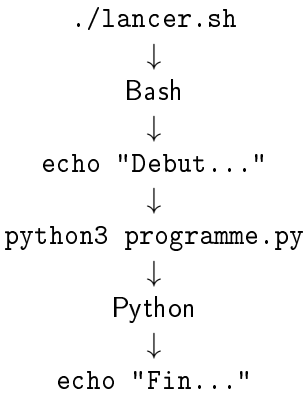
echo "Fin du traitement"
```

Puis :

```
chmod +x lancer.sh
./lancer.sh
```

Que vient-il de se passer ?

Notre script Shell orchestre l'exécution :



Un script Shell peut lancer d'autres programmes, y compris des programmes écrits dans

11 — Des arguments jusqu'à Python

Nous pouvons utiliser un argument reçu par le script Shell pour lancer Python.

`lancer.sh` :

```
#!/bin/bash

nom=$1

echo "Lancement du programme Python..."

python3 programme.py "$nom"

echo "Programme termine."
```

Exécution :

```
./lancer.sh Alice
```

Récupérer l'argument en Python

Dans `programme.py` :

```
import sys

nom = sys.argv[1]

print(f"Bonjour {nom} depuis Python !")
```

Le script Shell exécute :

```
python3 programme.py Alice
```

Python récupère alors `Alice` avec :

```
sys.argv[1]
```

Du Shell jusqu'à Python

```
./lancer.sh Alice
        ↓
   $1 = Alice
        ↓
python3 programme.py Alice
        ↓
   sys.argv[1]
        ↓
    "Alice"
```

12 — Exercice : Shell + Python

Nous disposons du projet suivant :

```
projet/
  addition.py
  statistiques.py
  graphique.py
  run.sh
```

Les trois programmes Python vous sont fournis.

Votre travail

Écrire le script `run.sh` permettant d'exécuter automatiquement les trois programmes.

Utilisation attendue

Votre script doit pouvoir être lancé avec :

```
./run.sh donnees.txt
```

Il devra successivement :

1. afficher un message de début ;
2. lancer `addition.py` ;
3. lancer `statistiques.py` ;
4. lancer `graphique.py` ;
5. transmettre `donnees.txt` à chaque programme ;
6. afficher un message de fin.

Résultat attendu

L'exécution :

```
./run.sh donnees.txt
```

doit produire quelque chose ressemblant à :

```
=== Debut du traitement ===

1. Calcul...
...

2. Statistiques...
...

3. Generation du graphique...
...

=== Traitement termine ===
```

Challenge — Plusieurs arguments

Modifiez votre script pour qu'il accepte deux arguments :

```
./run.sh donnees.txt resultat.txt
```

Vous devez pouvoir utiliser :

- $1 pour le fichier d'entrée ;
- $2 pour le fichier de résultat.

Affichez au début :

```
Fichier d'entree : donnees.txt
Fichier de sortie : resultat.txt
```

Les éléments à retenir

Élément	Signification
`#!/bin/bash`	interpréteur du script
`bash script.sh`	exécuter avec Bash
`chmod +x`	ajouter le droit d'exécution
`./script.sh`	exécuter un script local
`echo`	afficher du texte
`nom="Alice"`	définir une variable
`$nom`	valeur d'une variable
`$1`	premier argument
`$2`	deuxième argument
`$#`	nombre d'arguments
`python3 x.py`	exécuter un programme Python

Ce que nous savons maintenant faire

Nous sommes partis d'une commande exécutée manuellement :

```
python3 programme.py
```

pour arriver à :

```
./run.sh donnees.txt
```

Notre script peut maintenant :

- recevoir des informations ;
- stocker des valeurs dans des variables ;
- exécuter plusieurs commandes ;
- lancer des programmes Python ;
- transmettre des arguments à ces programmes.

Prochaine étape